

SPRFST Studio

Studio is the editor that comes with the language. It is a native Mac application —AppKit, Apple Silicon, no web view anywhere —and it drives the same `sprfst` toolchain you have been using from the terminal. Everything it shows you comes from the compiler, so the editor can never disagree with the build.

The window

```
+-----+-----??
?-----+-----??
?----+
| Explorer      | tabs                                | Outline  |
| Problems      |                                     |          |
| Outline       | editor, gutter, minimap                       | Minimap  |
| Git           |                                     |          |
| Debugger      |                                     |          |
+-----+-----??
?-----+-----??
?----+
| Terminal · Run · Playground                        |
+-----+-----??
?-----+-----??
?----+
| project · branch · line:column · timing · memory   |
+-----+-----??
?-----+-----??
?----+
```

Deep black behind, near black panels, warm orange for anything that matters, soft white text. Nothing blinks and nothing bounces.

Shortcuts worth knowing

Keys	Does
Cmd-P	command palette — every command, and every symbol in the file
Cmd-R	run the project
Cmd-B	build
Cmd-U	run the tests
Cmd-D	start the debugger
Cmd-\	toggle a breakpoint on this line
Ctrl-Command	format the file
Ctrl-Command	go to definition
Ctrl-Command	find everywhere
Shift-Command	open a folder

Shift-Command-N project

Ctrl-\ focus the terminal

Command-0 open this guidebook inside Studio

The panels

Explorer is the project tree. **Problems** lists every error and warning the compiler reported, with the code (E0201), the message and the position; clicking one jumps there. **Outline** lists the modules, types, functions and tests in the current file. **Git** shows the branch, the changed files and a diff, and can stage and commit. **Debugger** has the breakpoint list, the call stack, the variables and the watches, driven by the same engine as `sprfst debug`.

The bottom strip

Terminal is a real shell in the project folder — `sprfst`, `git`, anything. **Run** shows a program's output with its exit code, run time and peak memory. **Playground** runs the current file every time you stop typing and reports the output, the first error, the time it took and the memory it used, which makes it a fast way to try an idea without saving anything.

The editor

Syntax highlighting comes from the compiler's own lexer through the `tokens` request, so a new keyword is coloured the day it exists. Completion, hover and go-to-definition all ask the toolchain as well:

Behaviour	Request behind it
autocomplete	<code>complete <file> <offset></code>
hover documentation	<code>hover <file> <offset></code>
go to definition	<code>define <file> <offset></code>
rename	<code>rename <file> <offset></code>
format on save	<code>format <file></code>
problems	<code>check <file></code>
outline	<code>outline <file></code>

There is a minimap, a breakpoint gutter you can click, inline error marks, and bracket and indentation handling that understands SPRFST blocks.

Fonts

Studio ships four choices, in Settings:

 **SPRFST Hand** —the handwriting face used for the brand, for headings and for the welcome screen, with a readable code face for the code itself

 **Clean Code** —the same code face everywhere

`\225System` —the system interface font

`\225Monospace` —a fixed width face throughout

Size is adjustable, and the choice is remembered between launches.

Running the toolchain yourself

Studio finds `sprfst` in this order: inside its own bundle, then `/usr/local/bin`, then your `PATH`. The status bar shows the version it found. If it cannot find one, every command that needs it says so plainly instead of failing silently.

The protocol is open —`sprfst studio` on the command line starts the same service Studio talks to, one JSON request per line. Chapter 27 lists the requests.

Building Studio

On a Mac with Xcode's command line tools:

```
make app      # builds build/bin/sprfst and dist/SPRFST Studio.app
make dmg      # wraps the app in dist/SPRFST-Studio.dmg
```

`tools/build_macos_app.sh` compiles the Swift sources with `swiftc`, assembles the bundle, writes `Info.plist` with the `.spf` file association, copies the compiler, the standard library, the examples and this guidebook into `Contents/Resources`, and installs the icon. `tools/make_dmg.sh` then stages the app next to a shortcut to `/Applications` and builds a compressed, verified disk image with `hdiutil`.

Both scripts need macOS: `swiftc`, `AppKit` and `hdiutil` exist nowhere else. The compiler, the standard library, the examples and the guidebook build and run on any Unix.

Exercise

Open this repository in Studio, press `Cmd-T` type `guide`, and read chapter one from inside the editor. Then put a breakpoint in `examples/05-functions.spf` and press `Cmd-D`